

Generating Music with Machine Learning

Mini-Project #22 (UE24CS352A) | Nagashri R Patil (PES1UG24CS289), Nallamalli Kanaka Mani Sai Akhil (PES1UG24CS290)

1. Problem Statement

Following the Stanford project *Music Composition with Machine Learning* (Kang, Kim, Ringdahl, 2018), we learn musical structure from MIDI files and generate new melodies with accompanying chords. Input: sequences of note events from MIDI. Output: (i) a prediction of the next note and its duration, scored against the true next note on unseen songs, and (ii) newly generated, playable MIDI pieces. We compare an LSTM (main model), a Naive-Bayes-like chord→note model and a random (flat) baseline, as in the paper.

2. Dataset

Nottingham folk MIDI: 1034 MIDI files found, 1031 usable songs (3 skipped: too short/unparsable), 195,537 melody events. Split **by song** (80/10/10, seed 42): 824 train / 103 validation / 104 test songs (159,284 / 17,333 / 18,920 events). Files are read from data/; no dataset is hard-coded, so the same code runs on piano-midi.de, MAESTRO, etc. (The paper used 771 classical piano pieces for its encoder-decoder and 24 Chopin etudes for its LSTM.)

3. Approach

Representation (as in the paper). Each song is a series of note events. Per event we store the right-hand note (highest key $\geq$ MIDI 60, mapped to one of 88 piano keys), its duration (inter-onset interval quantised to {1,2,3,4,6,8,12,16} sixteenth-notes) and the left-hand chord (pitch-class set of notes < 60 sounding at that onset, top-63 chords kept, rest = <other>). **Random baseline:** uniform over 88 keys, 8 durations, all chords. **Naive-Bayes-like:** each note is independent given the chord, $P(\text{note}|\text{chord})$ and $P(\text{duration}|\text{chord})$ from Laplace-smoothed counts (smoothing tuned on validation); generation samples a chord path from a first-order Markov chain and then notes from $P(\cdot|\text{chord})$. It is evaluated both with the true concurrent chord (paper protocol, extra information) and with the previous chord (fair next-event task). **LSTM (main):** embeddings of the previous event's pitch/duration/chord $\rightarrow$ one LSTM layer (256 units) $\rightarrow$ dropout 0.3 $\rightarrow$ three softmax heads (next pitch, duration, chord); cross-entropy loss, Adam (lr 0.005, $\beta_1=0.9$, $\beta_2=0.999$), gradient clipping, early stopping on validation loss.

4. Implementation

Python/PyTorch (CPU, auto-uses CUDA). preprocess.py (pretty_midi parsing, split, vocab) $\rightarrow$ train.py (64-event windows, validation, checkpoint) $\rightarrow$ evaluate.py (test metrics, generation metrics, plots, MIDI export + reload check) $\rightarrow$ demo.py (generate/play a new MIDI). run_all.py runs all steps. LSTM trained 5 epochs (best epoch 5, 46s on cpu). Generation: 50 sequences of 128 events per model (temperature 1.0); every exported file re-loads correctly (Random (flat): 5/5, Naive Bayes-like: 5/5, LSTM: 5/5).

5. Results

Table 1 - next-event prediction on 17,093 held-out test events (CE = cross-entropy in nats, lower is better).

Model	Pitch CE	Pitch ppl	Top-1	Top-5	Dur. CE	Dur. acc
Random (flat)	4.477	88.0	0.000	0.002	2.079	0.165
Naive Bayes-like (prev chord)	2.445	11.5	0.179	0.664	0.990	0.655
Naive Bayes-like (given chord)	2.305	10.0	0.221	0.719	0.993	0.655
LSTM	1.652	5.2	0.430	0.900	0.713	0.742

Table 2 - objective metrics of generated music (mean over generated sequences; closer to the Real column is better).

Metric	Real (test)	Random	Naive Bayes	LSTM
Scale consistency	0.98	0.65	0.92	0.97
Chord-tone rate (harmony)	0.75	0.28	0.74	0.64
Repeated-note rate	0.11	0.01	0.13	0.11
Mean interval (semitones)	2.62	29.58	5.36	2.60
Leaps > 1 octave	0.00	0.73	0.05	0.00
5-gram novelty vs train	0.26	1.00	0.86	0.37
Interval-histogram JS divergence vs real (lower = closer)	0	0.65	0.17	0.00

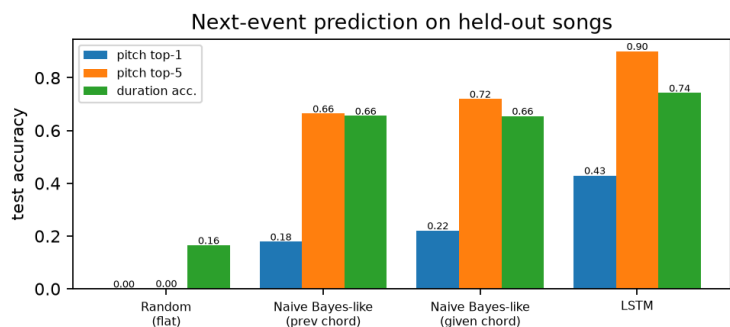
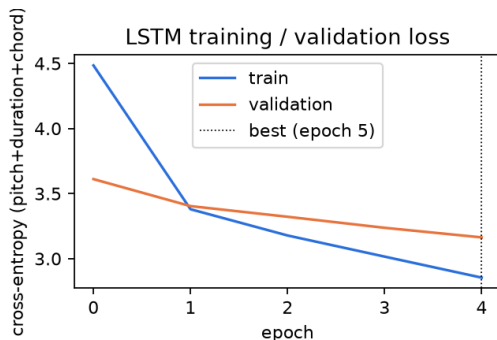


Fig. 1 LSTM training/validation loss. Fig. 2 Test accuracy of the three models.

Discussion. On unseen songs the LSTM predicts the next pitch with CE 1.652 (perplexity 5.2) versus 2.445 for the Naive-Bayes-like model with the previous chord and 4.477 for the flat baseline (top-1 accuracy 0.43 vs 0.18 vs 0.00). Even when the Naive-Bayes-like model is handed the true concurrent chord, its pitch CE is 2.305. Generated LSTM music has an interval distribution closest to real music (JS 0.00 vs 0.17 for Naive Bayes and 0.65 for random), with scale consistency 0.97 (real 0.98) and a 5-gram novelty of 0.37 (real test music: 0.26), i.e. it reuses training phrases no more than real held-out music does. On harmony the picture is mixed: the chord-tone rate is 0.74 for Naive Bayes (which is built on $P(\text{note}|\text{chord})$), 0.64 for the LSTM and 0.75 for real music. Real test music itself has only 0.26 5-gram novelty against the training set, so near-duplicate tunes exist across splits and absolute scores are optimistic. These numbers are objective proxies; the paper additionally ran a human 'music Turing test', which we did not repeat.

6. Conclusions

An LSTM trained on MIDI note events (pitch, duration, chord) clearly outperforms a Naive-Bayes-like chord→note model and a random baseline, reproducing the paper's finding that sequence modelling beats independent-note models. The Naive-Bayes-like model produces notes that fit the chord but has no melodic memory, so its melodies jump erratically; the random baseline is atonal.

Limitations: a single-voice melody representation (highest right-hand note), coarse chord vocabulary, no velocity/dynamics, and no human listening study; results depend on the chosen dataset (ours: Nottingham folk MIDI; the paper used classical piano). **Future work:** Transformer/self-attention models and velocity, as suggested in the paper's future work. Code, README and generated MIDI: GitHub repository (*placeholder: github.com/<username>/music-generation-ml*).